

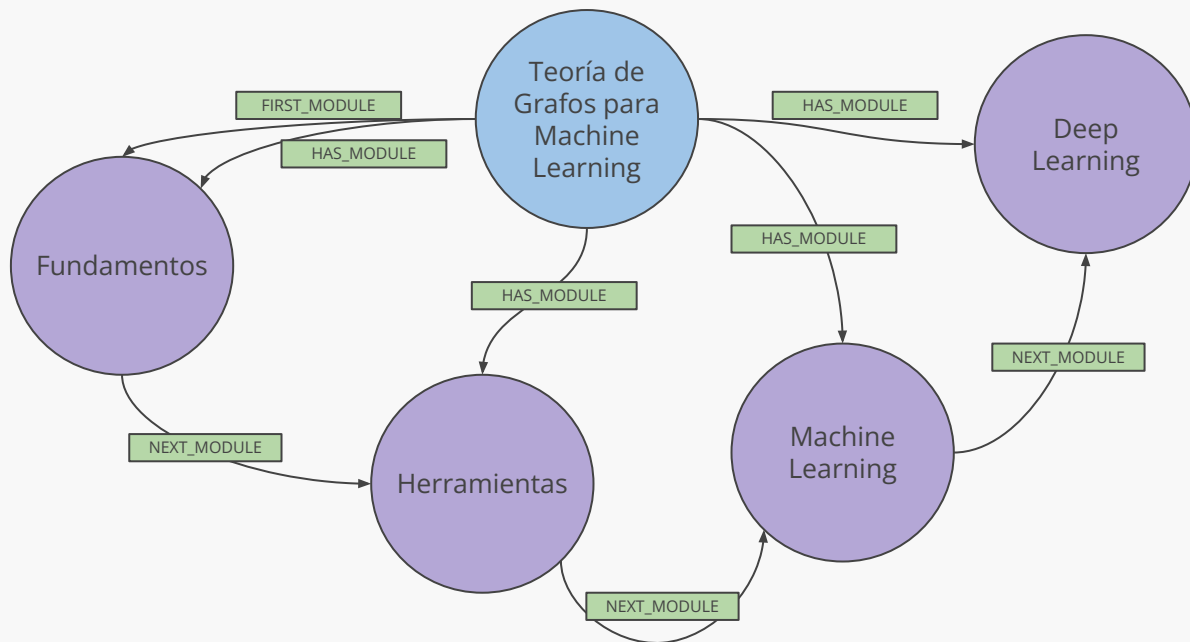
Clase 6.1

Deep learning para grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

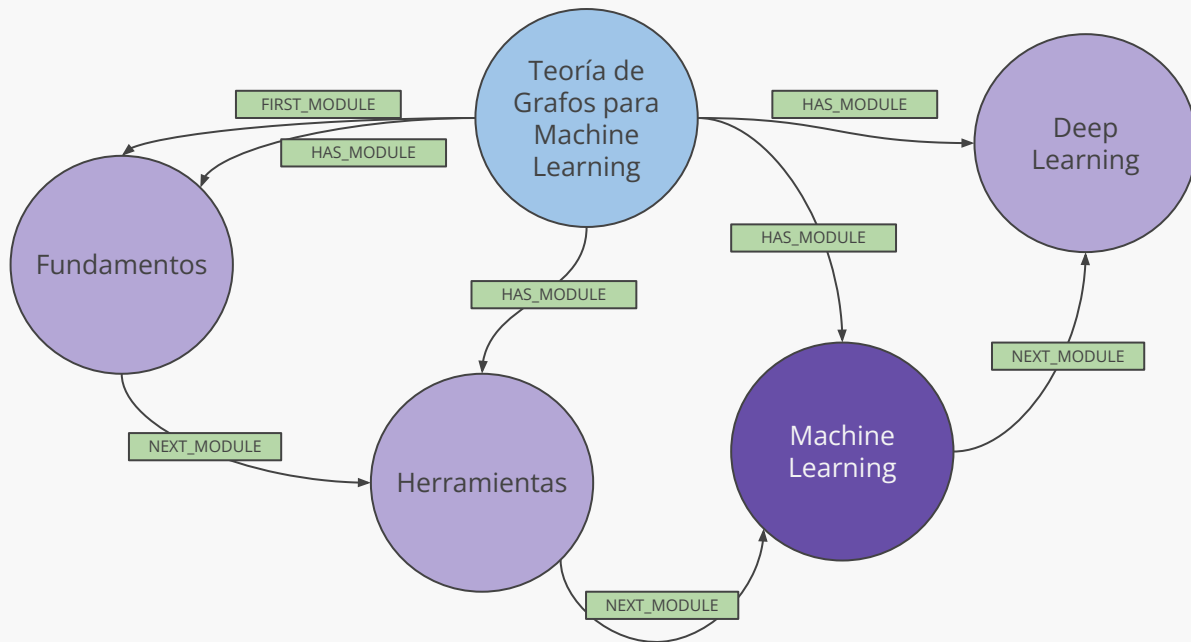


Contenidos del curso

3. Machine Learning

A partir de analítica de grafos, construir features que podamos usar en modelos de ML...

... con el objetivo de entender cómo se aplican conceptos de ML sobre grafos

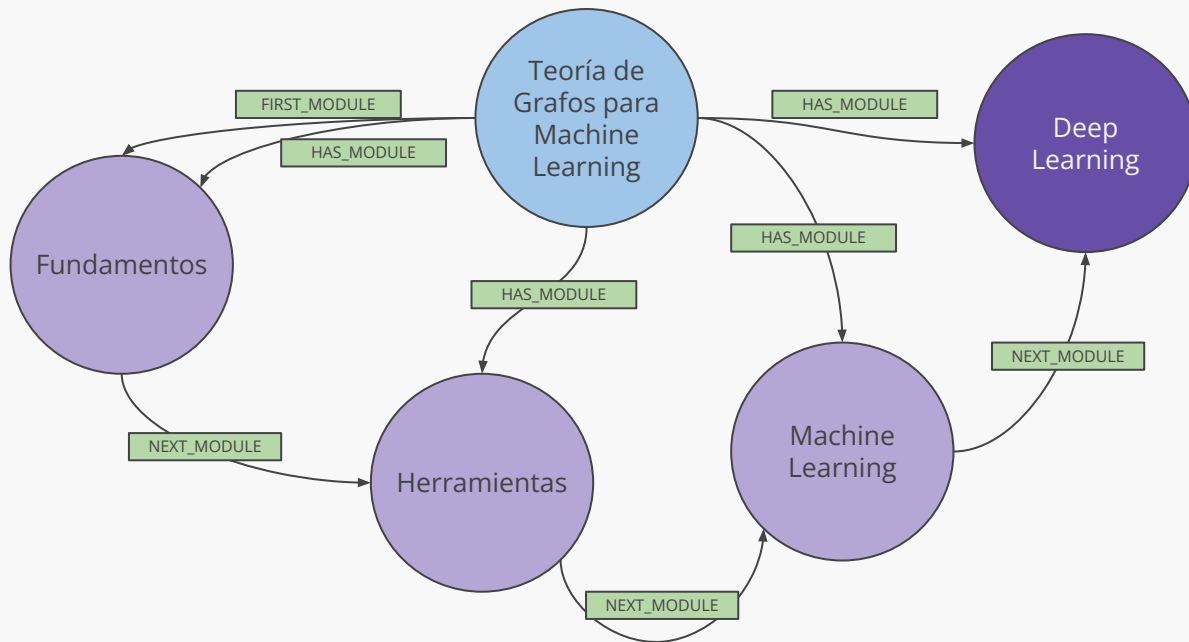


Contenidos del curso

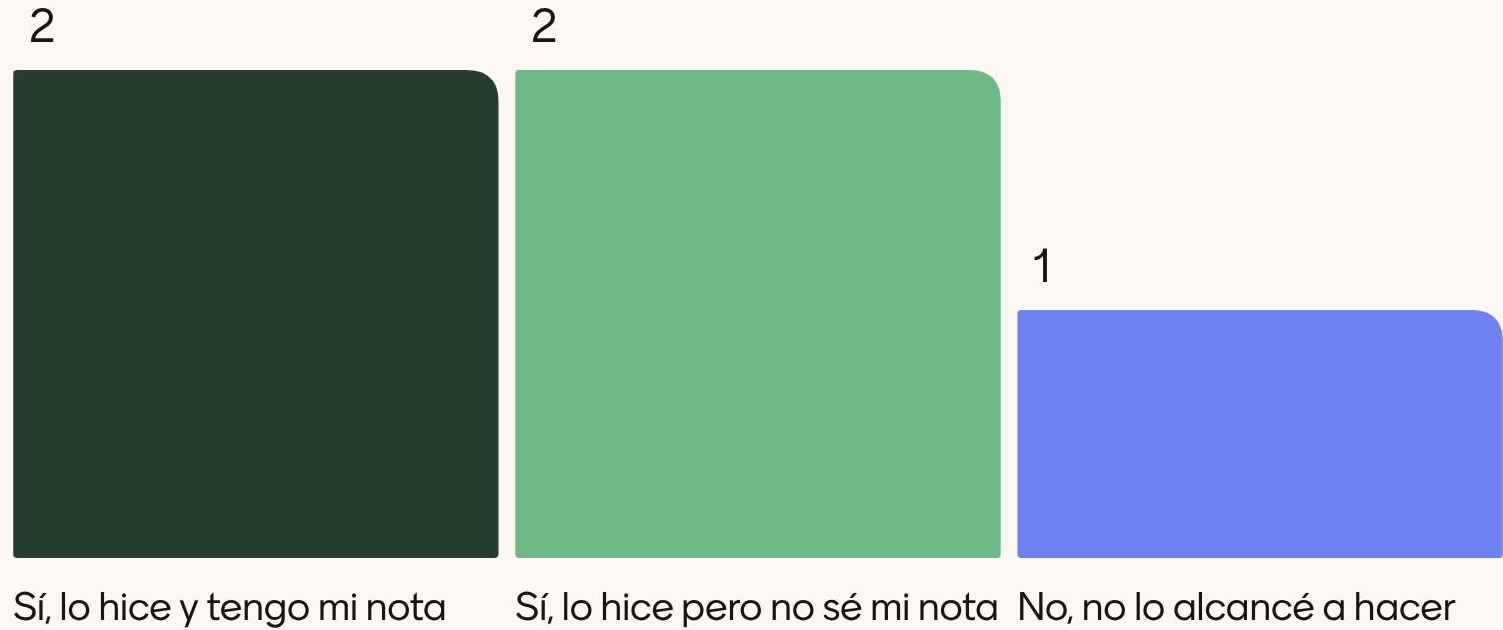
4. Deep Learning

Explorar cómo se extiende el concepto de ML para implementar modelos especializados en grafos (GNNs)...

... de manera de aprovechar lo más posible la estructura y naturaleza de un grafo



¿Hicieron el Control 2?



¿Qué te ha parecido el curso hasta el momento?

acorde a lo esperado

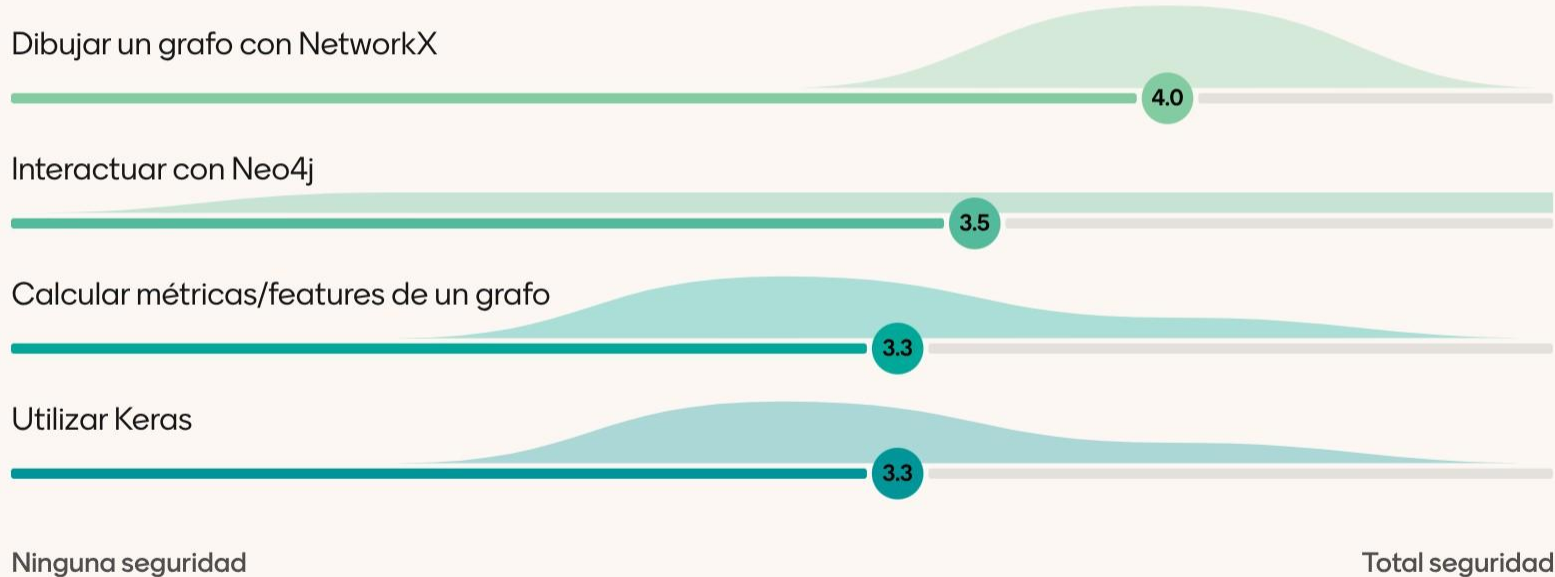
Muy bueno; únicamente
hubiera sido bueno ver
casos de uso para el ámbito
laboral

Muy interesante, pero me ha
sido difícil encontrar un
caso de uso para mi
realidad

Interesante. Una forma
nueva de modelar

Interesante . Pero se
nota que no es de la u

¿Qué tanta seguridad sientes para:



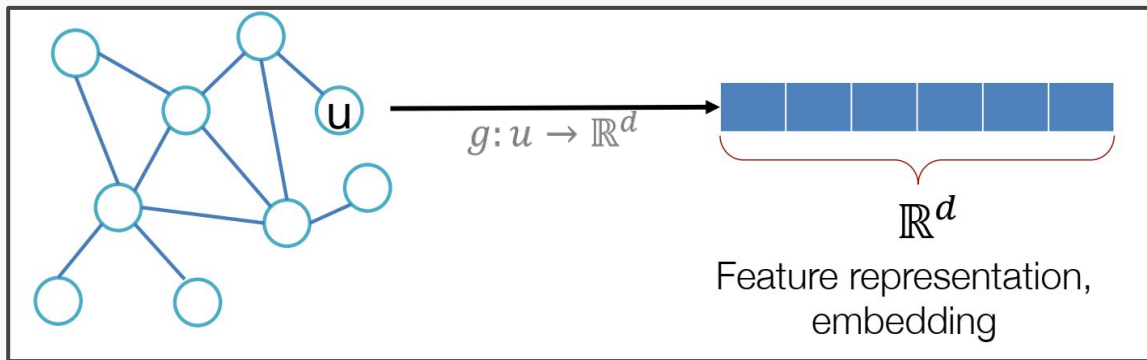
¡Gracias por sus respuestas!

Empecemos con la clase de hoy



Aprender la representación de un grafo

Objetivo: Aprender una representación independiente de la tarea en cuestión para aplicar ML sobre grafos

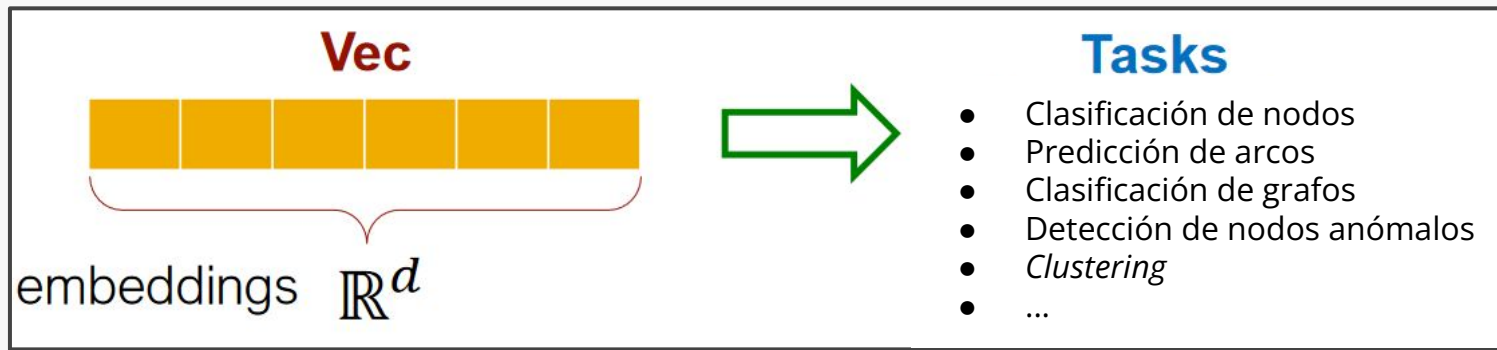


Podemos ver también esto como el aprendizaje de un mapeo de nodos a un espacio de embedding, donde nodos similares sean cercanos en esta espacio

¿Por qué usar *embeddings*?

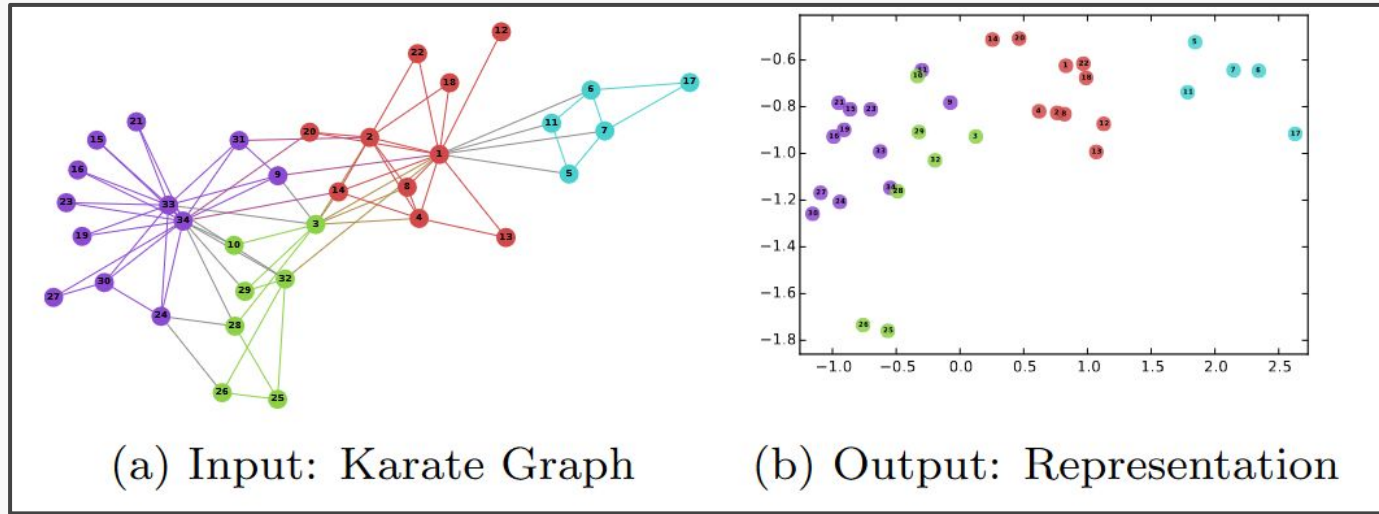
Tarea: Llevar nodos a un espacio latente (representados por un embedding):

- Similitud de *embeddings* indica similitud en grafo (ej: existe un arco)
- Codifican información del grafo
- Potencialmente usables en múltiples tareas

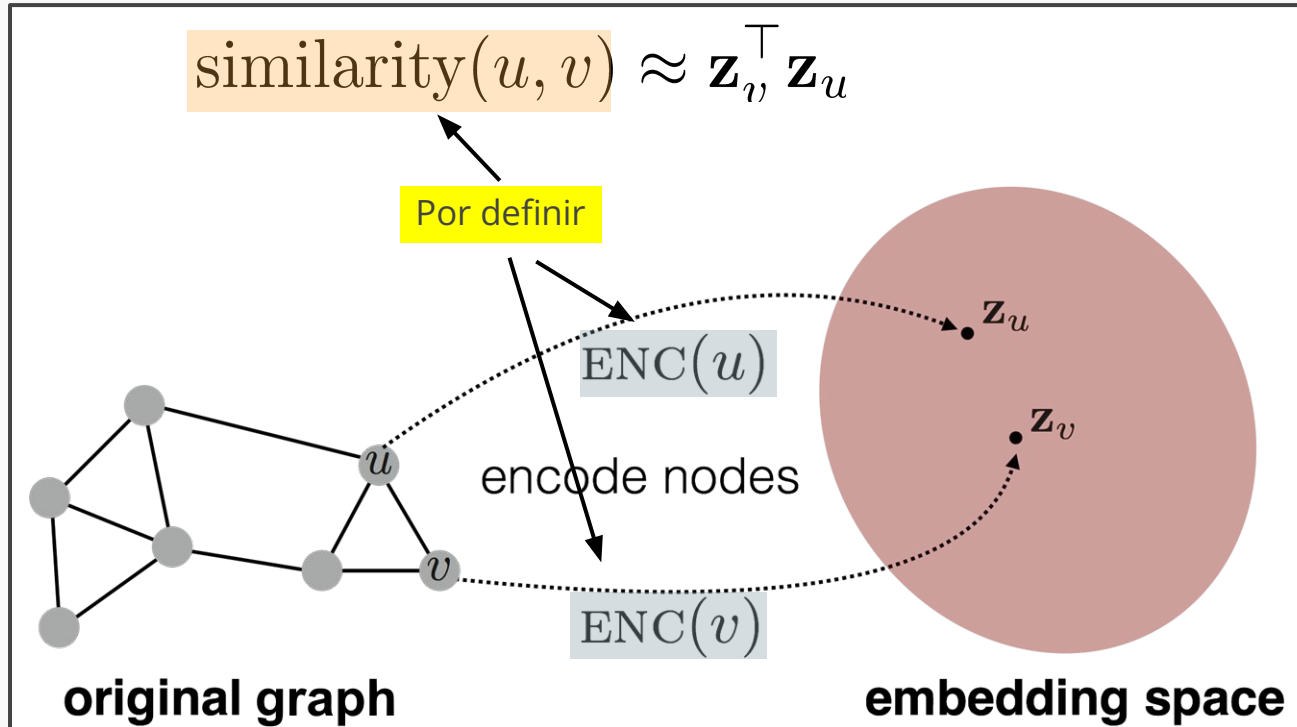


Ejemplo: Representar interacciones sociales

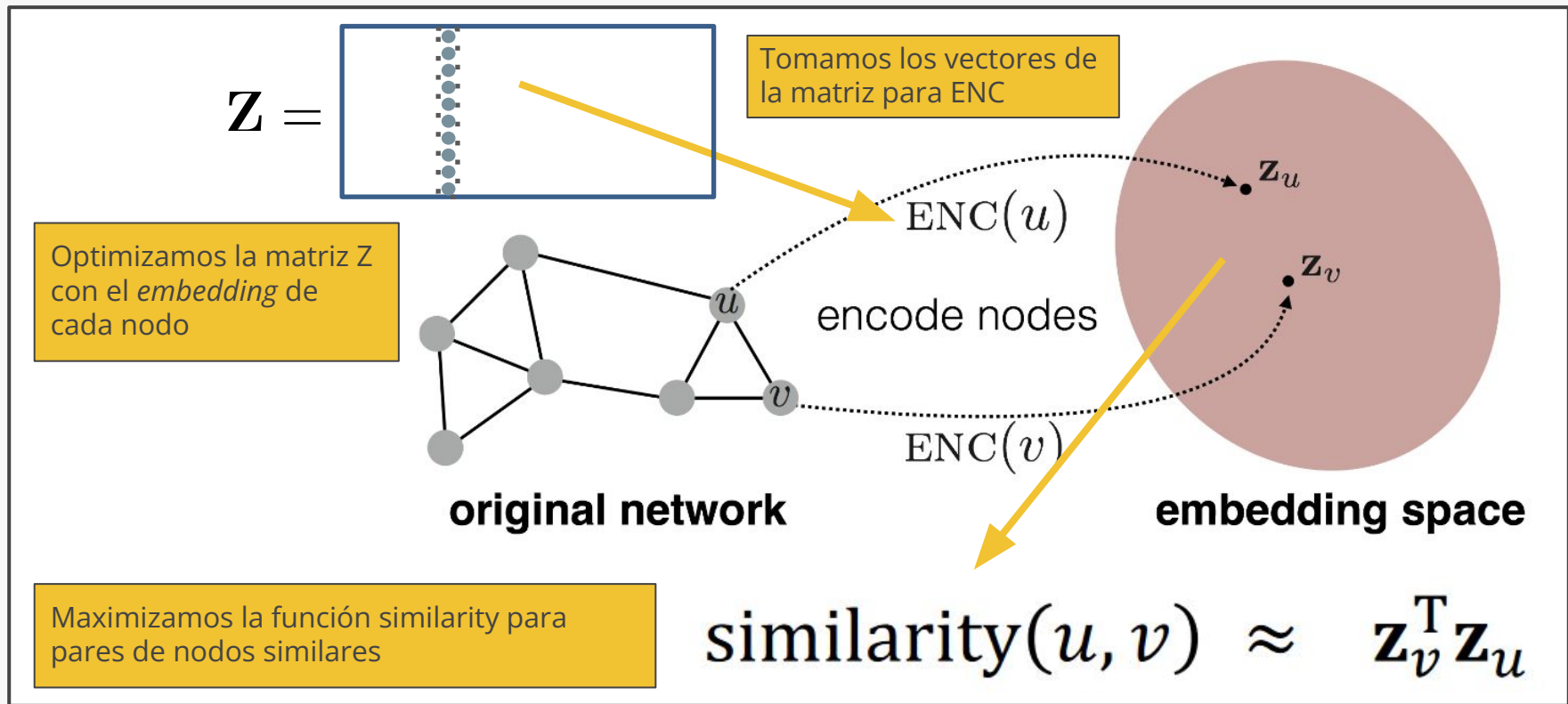
Podemos interesarnos en un espacio de embedding, donde nodos “similares” (según la estructura del grafo) se encuentren cerca en el espacio.



Embedding de nodos (*node embeddings*)



Resumen: Encoder + Decoder



Necesitamos estos dos elementos

- **Encoder ENC**: mapea cada nodo a un vector de baja dimensionalidad

$$\text{ENC}(v) = \mathbf{z}_v$$

Embedding
d-dimensional

Nodo del grafo

- **Función de similaridad similarity**: especifica cómo se mapean las relaciones del grafo original al espacio de embedding

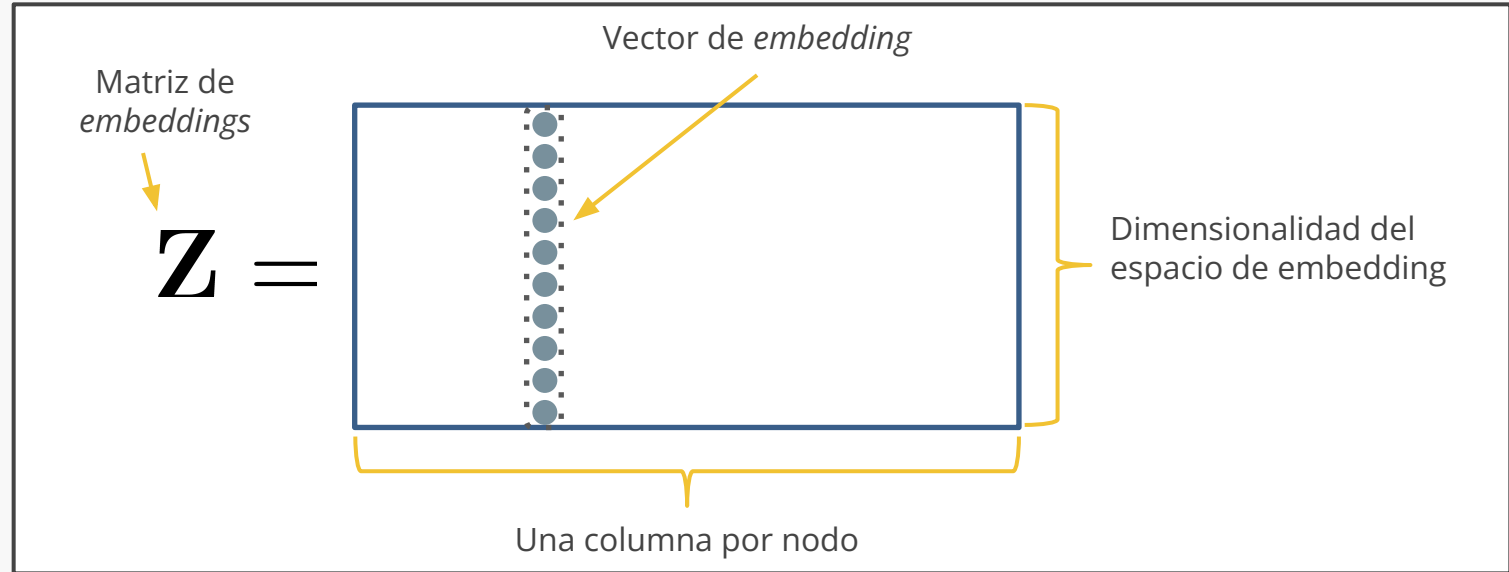
$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

Similitud entre u y v en el grafo

Producto punto entre *embeddings* de nodos

Encoding simple

El **encoder** más simple es simplemente indexar un vector en una matriz de *embeddings*, es decir, tomar las *features* desde un *embedding* ya definido:



Limitaciones de este *approach*

- **Necesita $O(|V|d)$ parámetros:**
 - No comparte parámetros entre nodos
 - Cada nodo tiene su propio *embedding*
- **No tiene capacidad de generalización**
 - No puede hacer *embeddings* para nodos no vistos durante el entrenamiento
- **No incluye *features* de los nodos**
 - Muchas veces los nodos tienen atributos que podemos y deberíamos usar

Pregunta#¿Podemos hacerlo mejor?

- **Necesita $O(|V|d)$ parámetros:**
 - No comparte parámetros entre nodos
 - Cada nodo tiene su propio *embedding*
- **No tiene capacidad de generalización**
 - No puede hacer *embeddings* para nodos no vistos durante el entrenamiento
- **No incluye *features* de los nodos**
 - Muchas veces los nodos tienen atributos que podemos y deberíamos usar

¿Dónde aplicaremos DL?

Usaremos métodos de *deep learning* basados en redes neuronales para grafos (GNNs)

$$\text{ENC}(v) = \text{Múltiples capas de transformaciones no lineales basadas en la estructura del grafo}$$

Nota: todos los *encoders* de esta clase pueden combinarse con las funciones de similaridad que ya hemos visto en el curso

Tipos de tareas

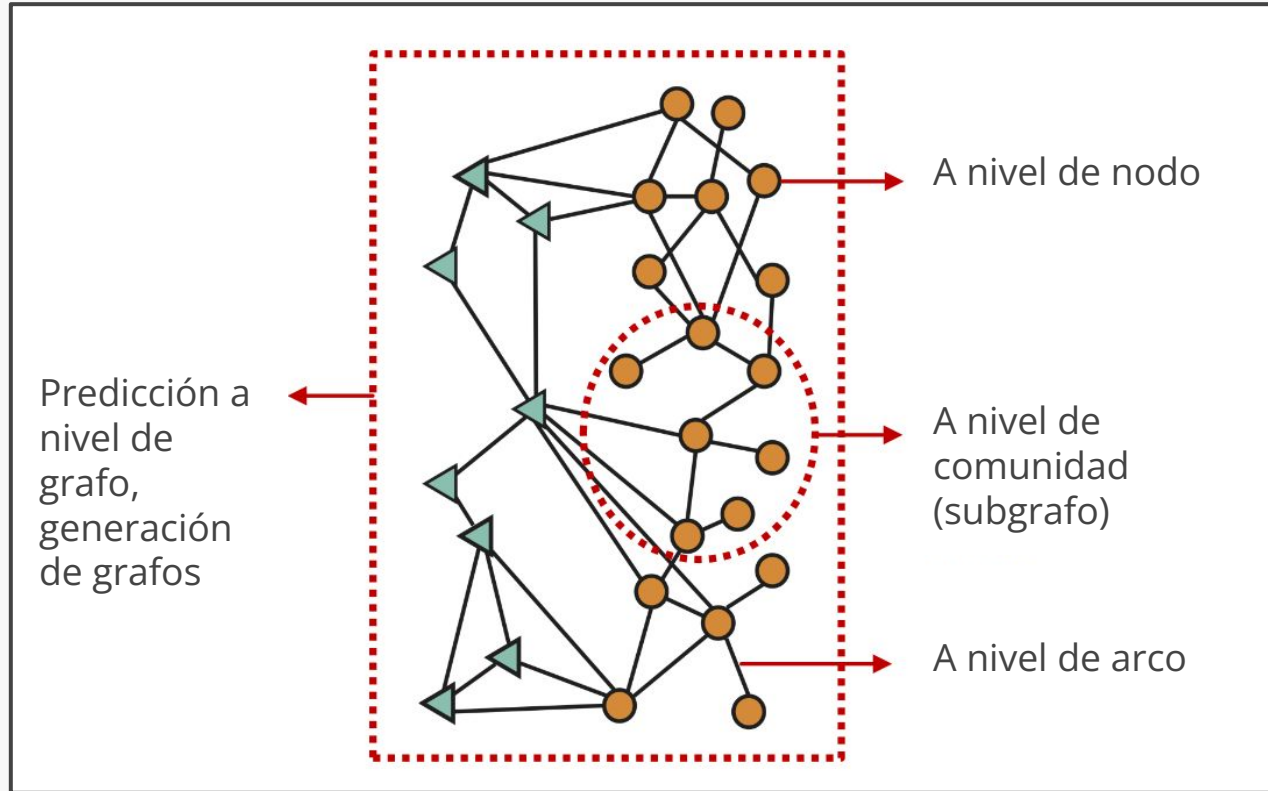
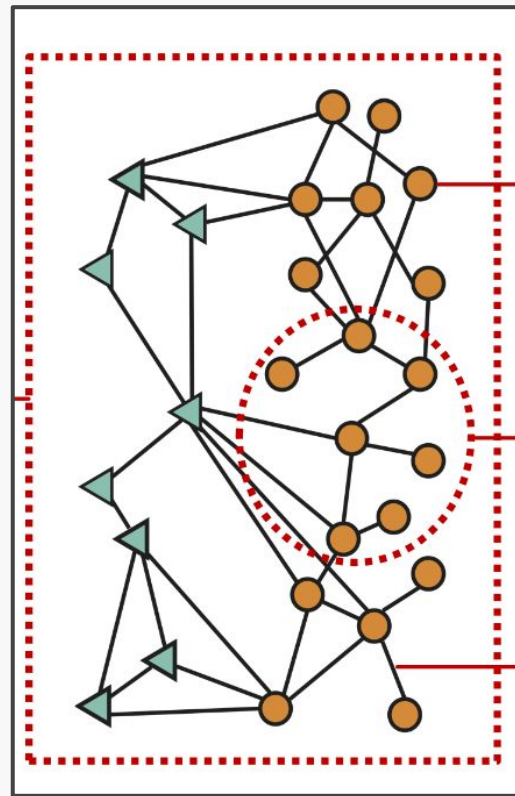


Figura: [CS224W: Machine Learning with Graphs \(Jure Leskovec\) @ Stanford](#)

Tipos de tareas

- **Clasificación de nodos:** Predecir una propiedad de un nodo (por ejemplo, categorizar usuarios o ítems en un contexto de recomendación)
- **Predicción de arcos:** Predecir si hay relaciones faltantes entre dos nodos (por ejemplo, completar un grafo de conocimiento)
- **Clasificación de grafos:** Categorizar diferentes tipos de grafos (por ejemplo, predecir propiedades moleculares)
- **Clustering:** Detectar si nodos forman una comunidades (por ejemplo, detección de grupos sociales)
- **Otras tareas:**
 - **Generación de grafos:** Descubrimiento de drogas
 - **Evolución de grafos:** Simulaciones físicas



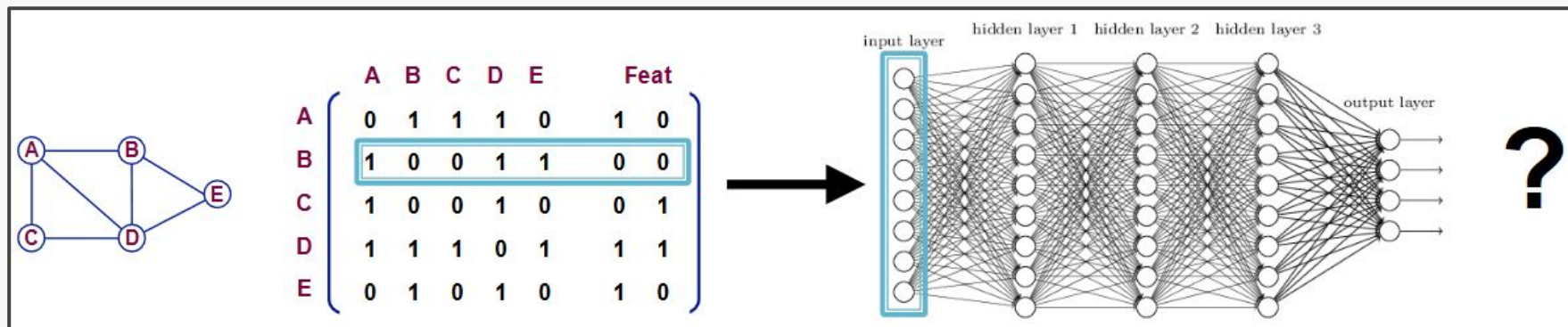
Formalización

Sea un grafo G :

- V es el conjunto de nodos
- A es la matriz de adyacencia
- $X \in \mathbb{R}^{|V| \times d}$ es una matriz de *features* de nodos
- v : es un nodo en V ; $N(v)$ es el conjunto de vecinos de v
- Features de nodos:
 - En una red social: perfil de usuario, foto de usuario
 - En una red biológica: perfiles de expresión génica, información funcional del gen
 - Cuando no hay features de nodos en el dataset de grafos:
 - Vectores indicadores (one-hot de un nodo)

Una primera idea

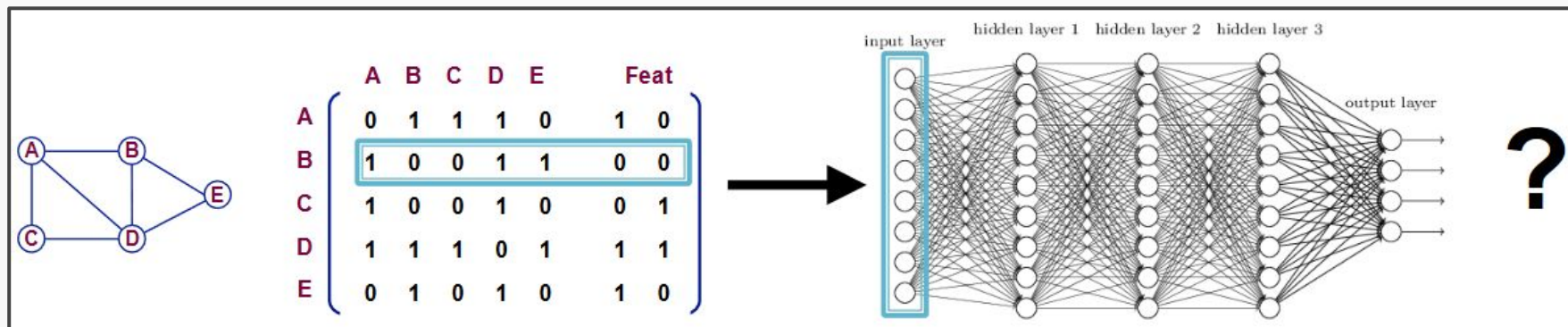
1. Concatenar matriz de adyacencia y matriz de features
2. Entregarla a una red neuronal



Una primera idea

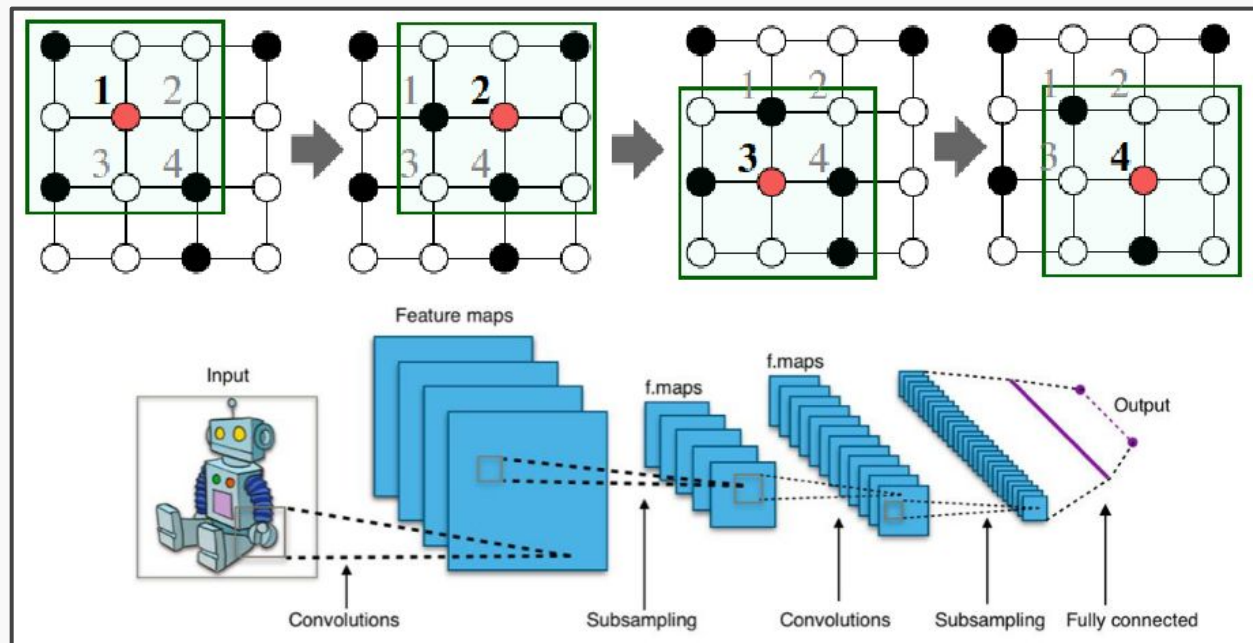
Problemas:

- $O(|V|)$ parámetros
- No aplicable a grafos de diferentes tamaños
- Sensible al orden de los nodos



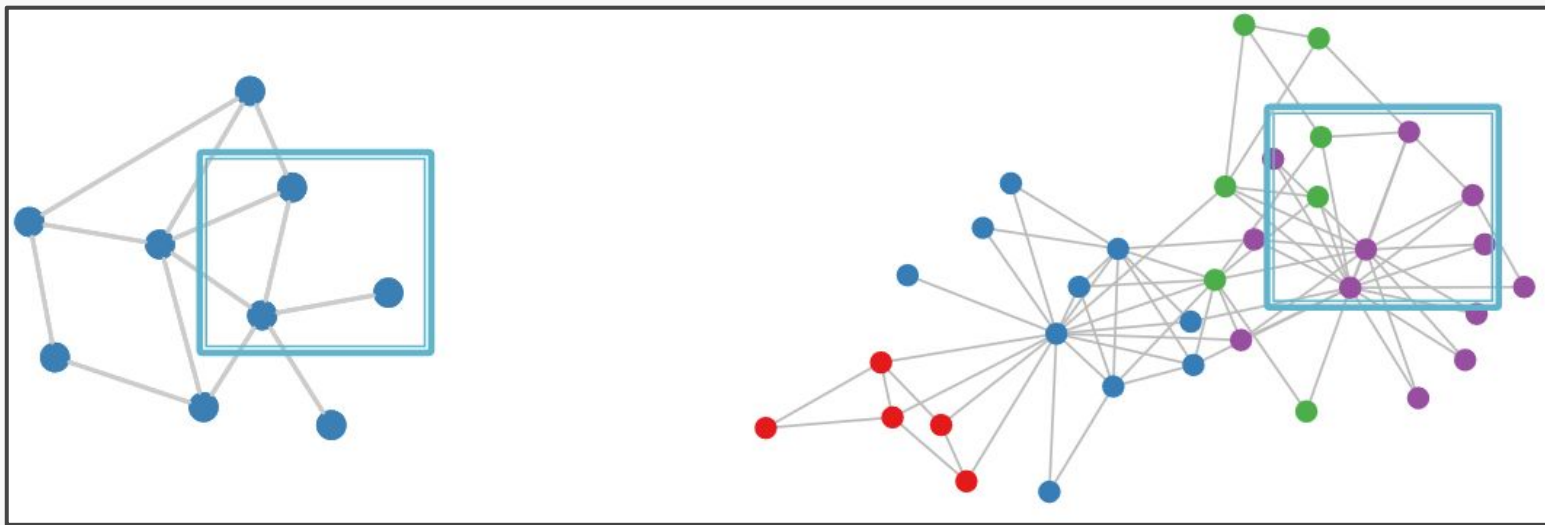
Otra idea: capas convolucionales

Buscamos generalizar convoluciones más allá de redes simples y considerar features de nodos

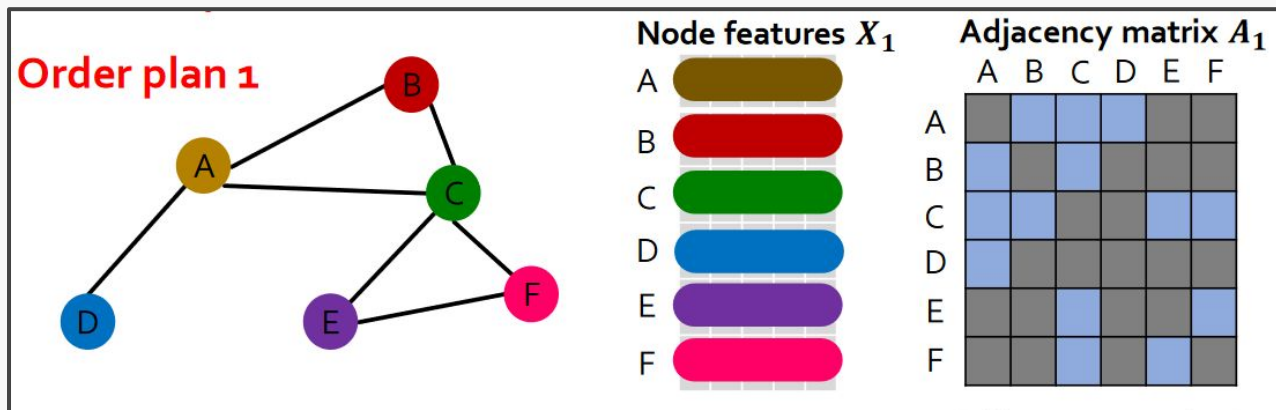


Grafos en el mundo real

No hay una noción de localidad o tamaño de kernel o ventana fijo en un grafo. Además, los grafos son invariantes en permutaciones (no tienen un orden de nodos)



Invariante a permutaciones



- *Permutation invariant*: Permutación en el input no afecta el output
- *Permutation equivariant*: Permutación en el input ocurre igual en output

Invariante a permutaciones

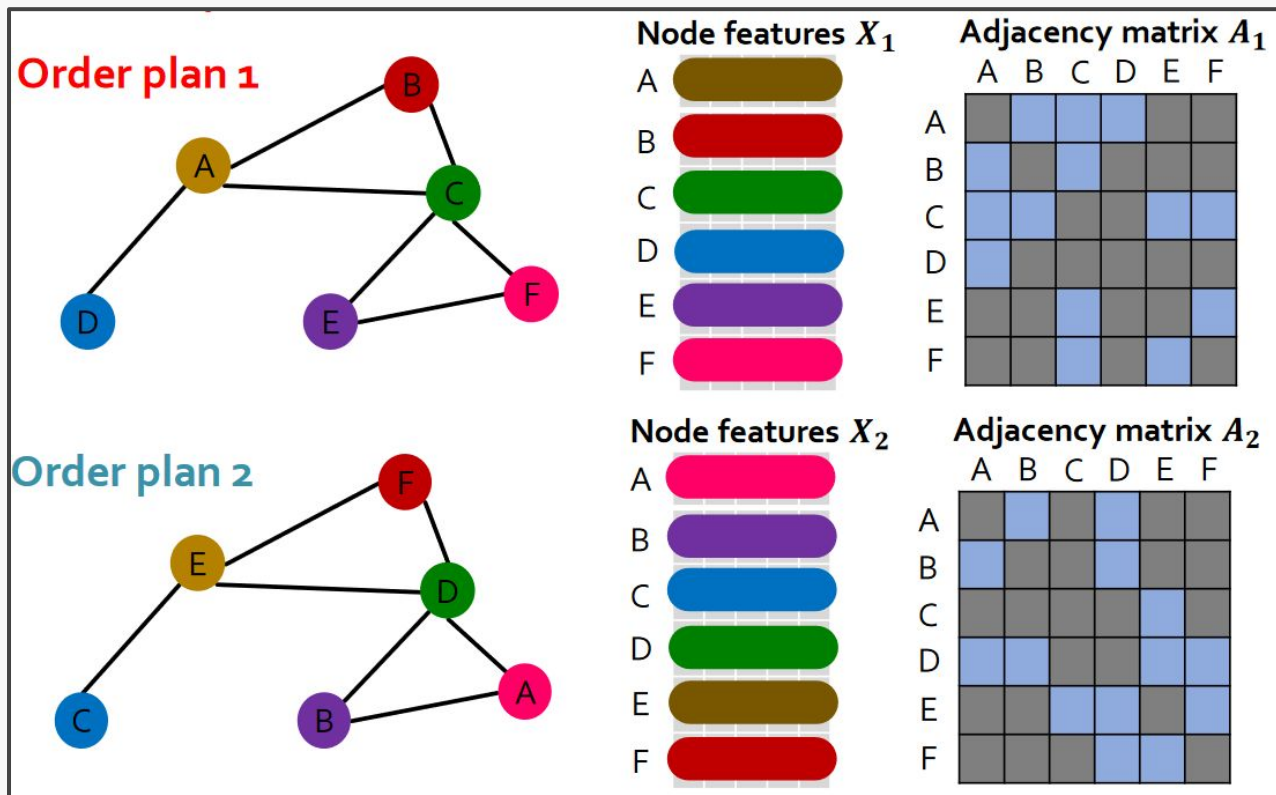
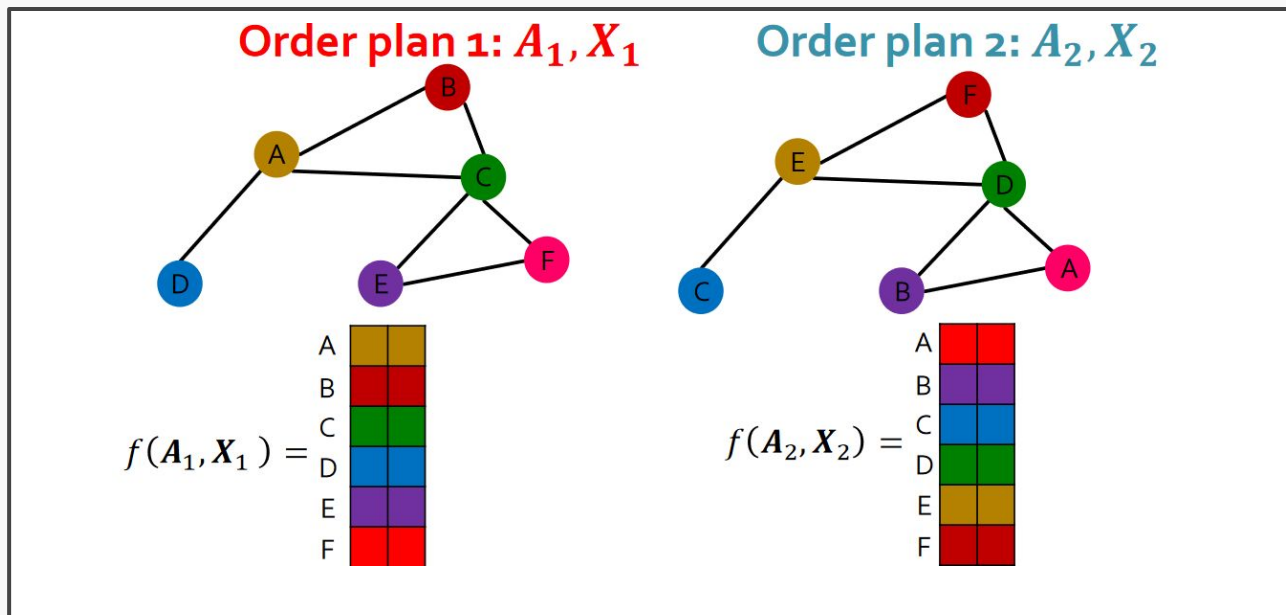


Figura: [CS224W: Machine Learning with Graphs \(Jure Leskovec\) @ Stanford](#)

Invariante a permutaciones

Para representación de nodos:

Aprendemos una función f que mapee nodos de G a una matriz de $R^{m \times d}$



Invariante a permutaciones

Para representación de nodos:

Aprendemos una función f que mapee nodos de G a una matriz de $R^{m \times d}$

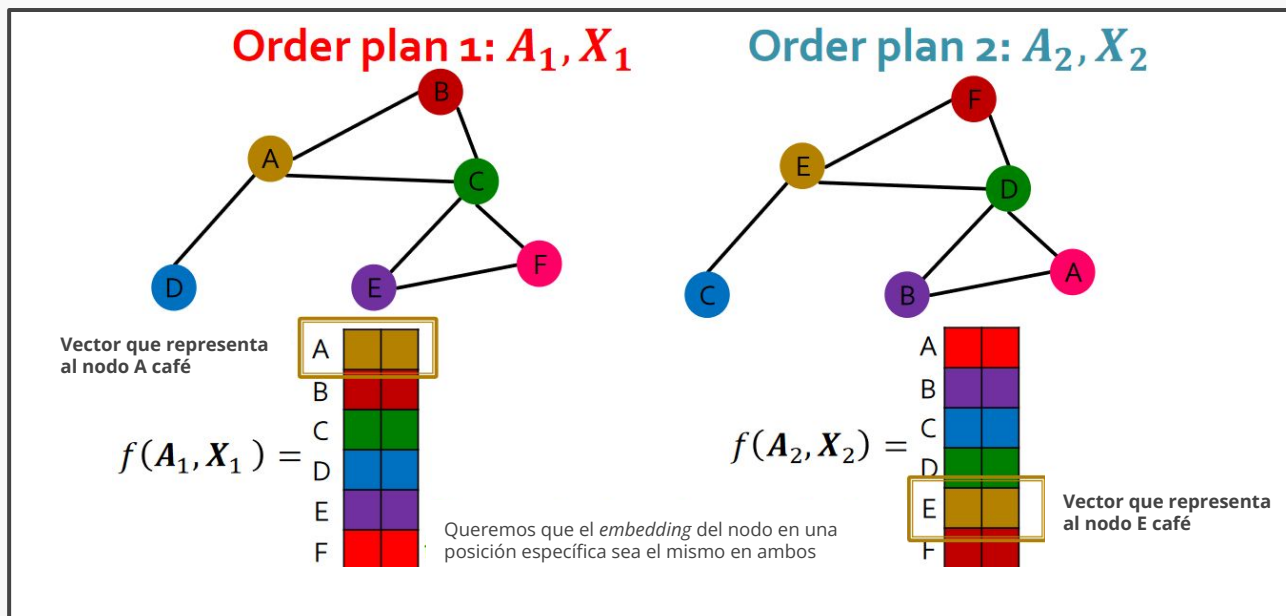
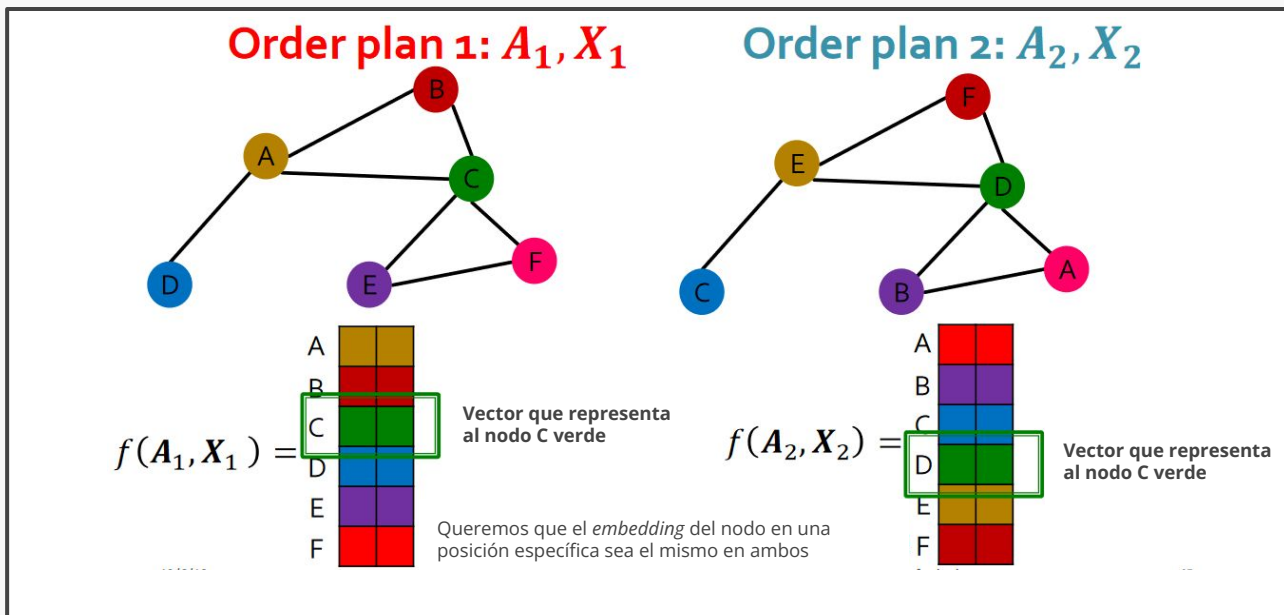


Figura: [CS224W: Machine Learning with Graphs \(Jure Leskovec\) @ Stanford](#)

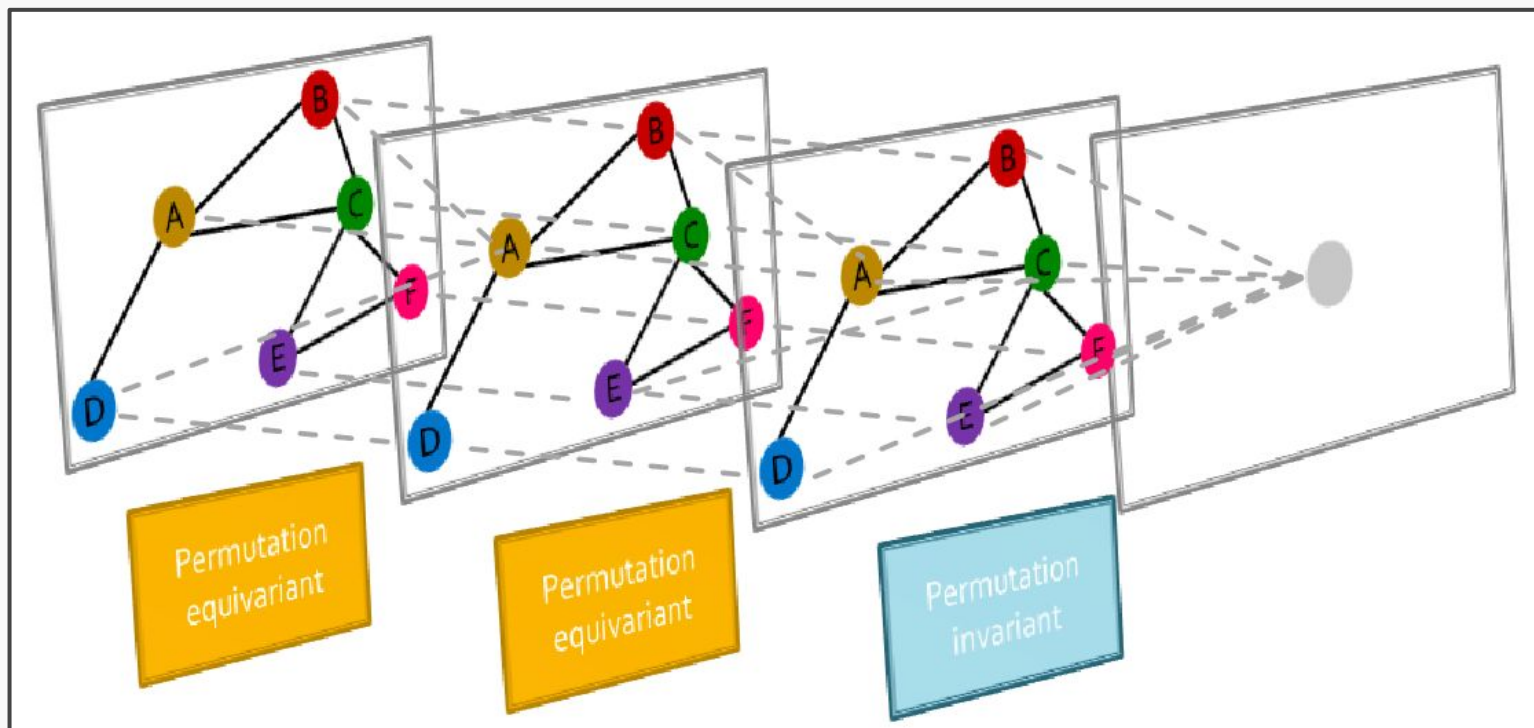
Invariante a permutaciones

Para representación de nodos:

Aprendemos una función f que mapee nodos de G a una matriz de $R^{m \times d}$

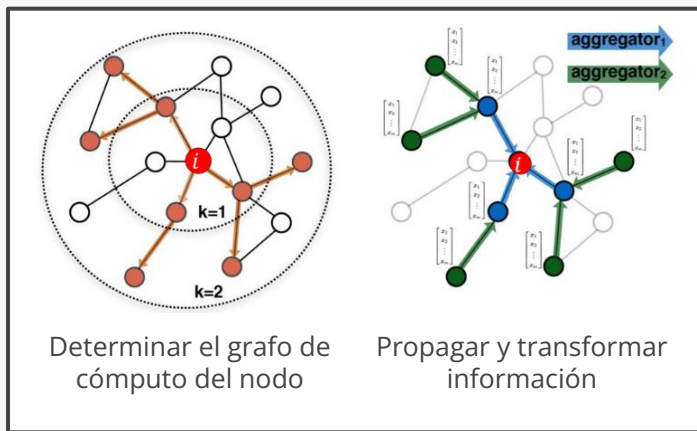


GNNs consisten en funciones equivariantes



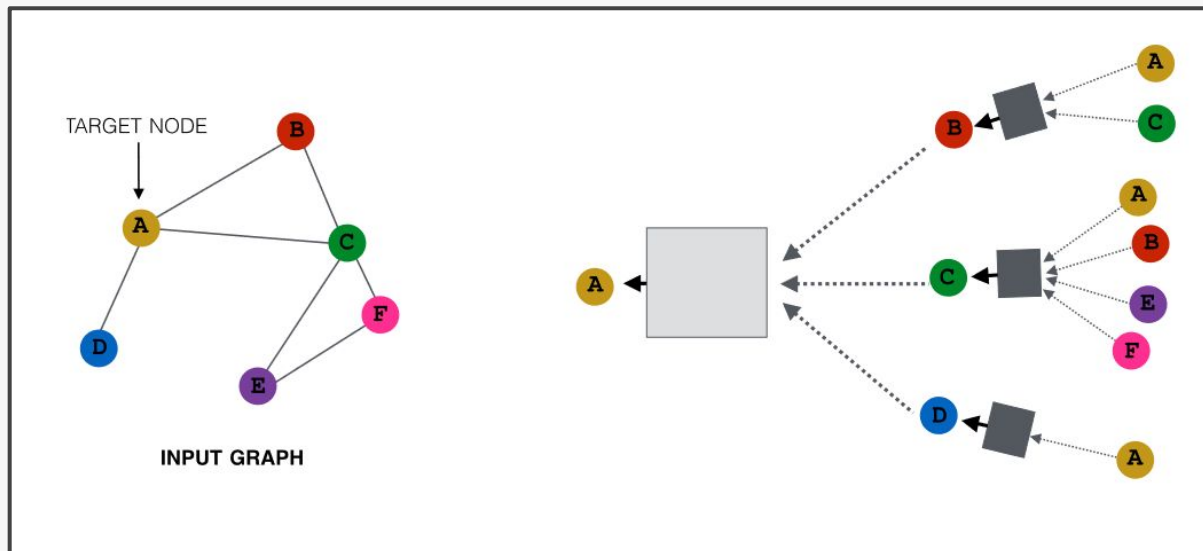
Graph Convolutional Networks

Idea: Los vecinos de un nodo definen el grafo de cómputo (es decir, cómo calcular la representación)



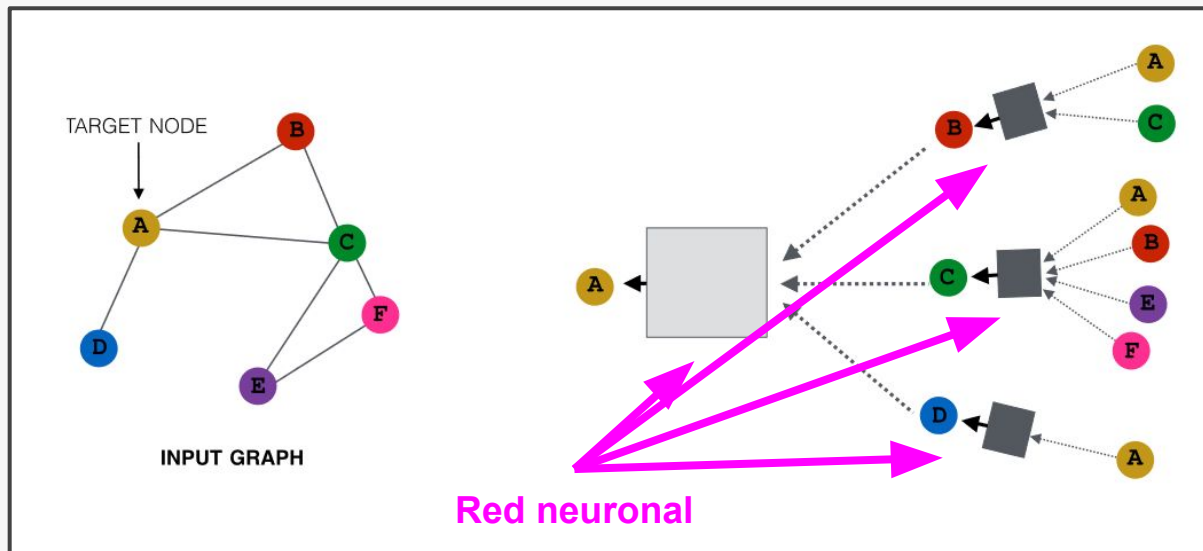
Estamos aprendiendo a propagar información a través del grafo para calcular las *features* de cada nodo

Idea: agregar información de vecinos



Idea: Construir *embeddings* de nodos a partir de vecindarios locales en grafo

Idea: agregar información de vecinos

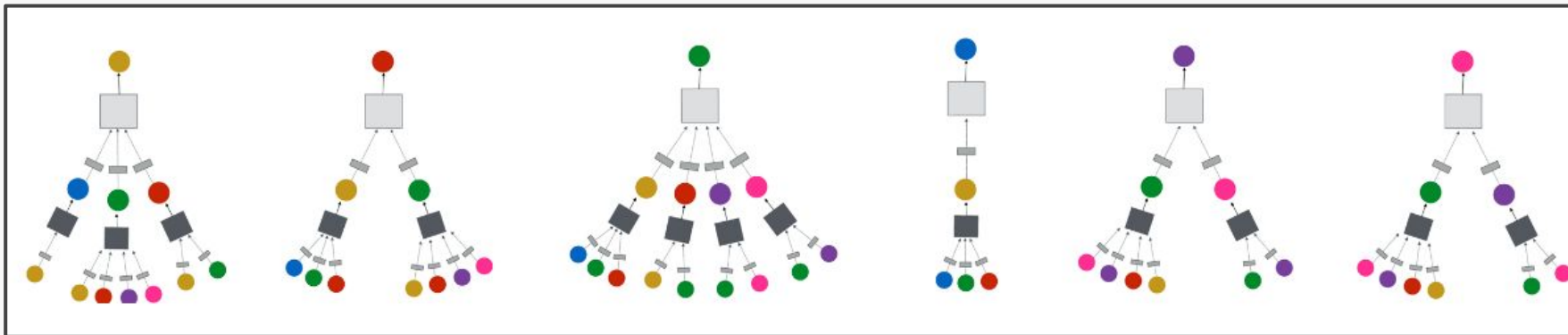
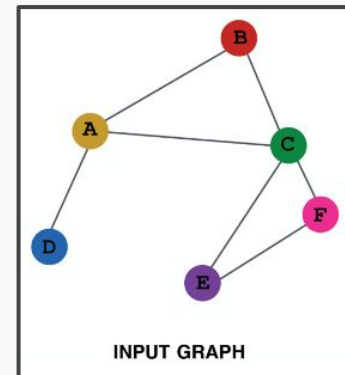


Intuición: Nodos agregan información de sus vecinos usando NNs

Idea: agregar información de vecinos

Intuición: Un vecindario del grafo define un grafo de cómputo

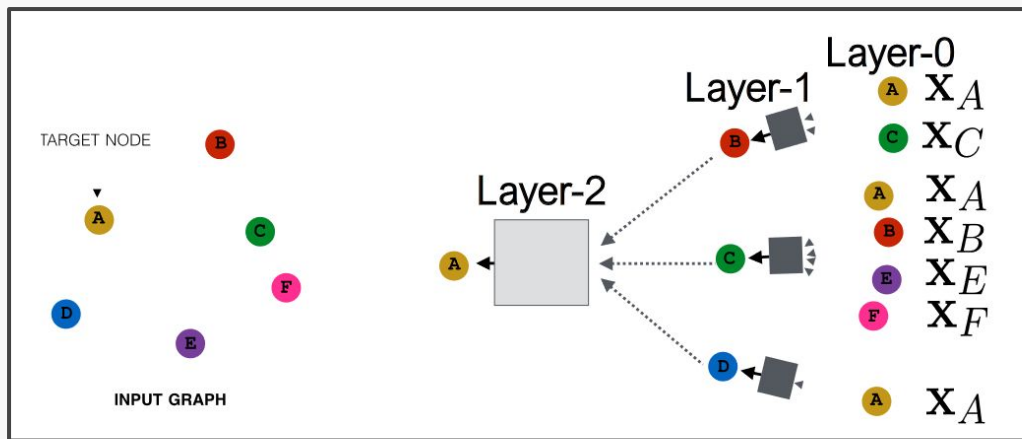
Cada nodo define un grafo de cómputo en base a su vecindario



Modelos *deep*, de muchas capas (DL)

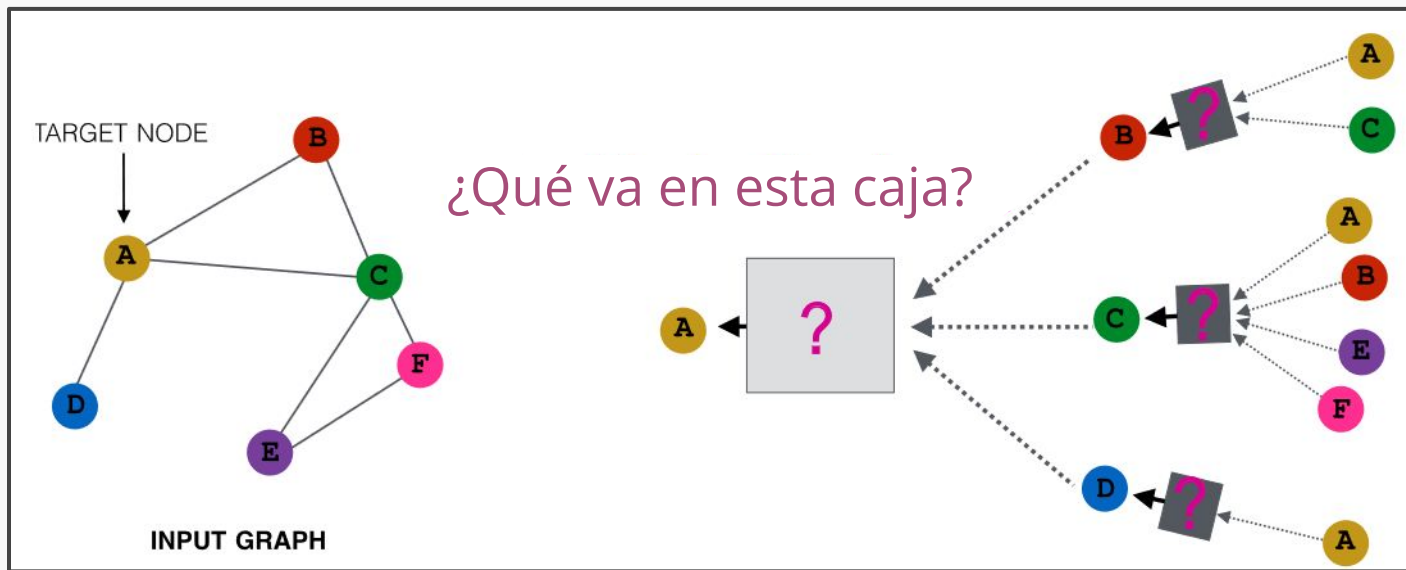
Podemos usar cualquier profundidad (número de capas):

- Nodos tienen *embeddings* en cada capa
- El *embedding* de la capa 0 para el nodo v es su *feature* de entrada, x_v
- El *embedding* de la capa k obtiene información de nodos a distancia k



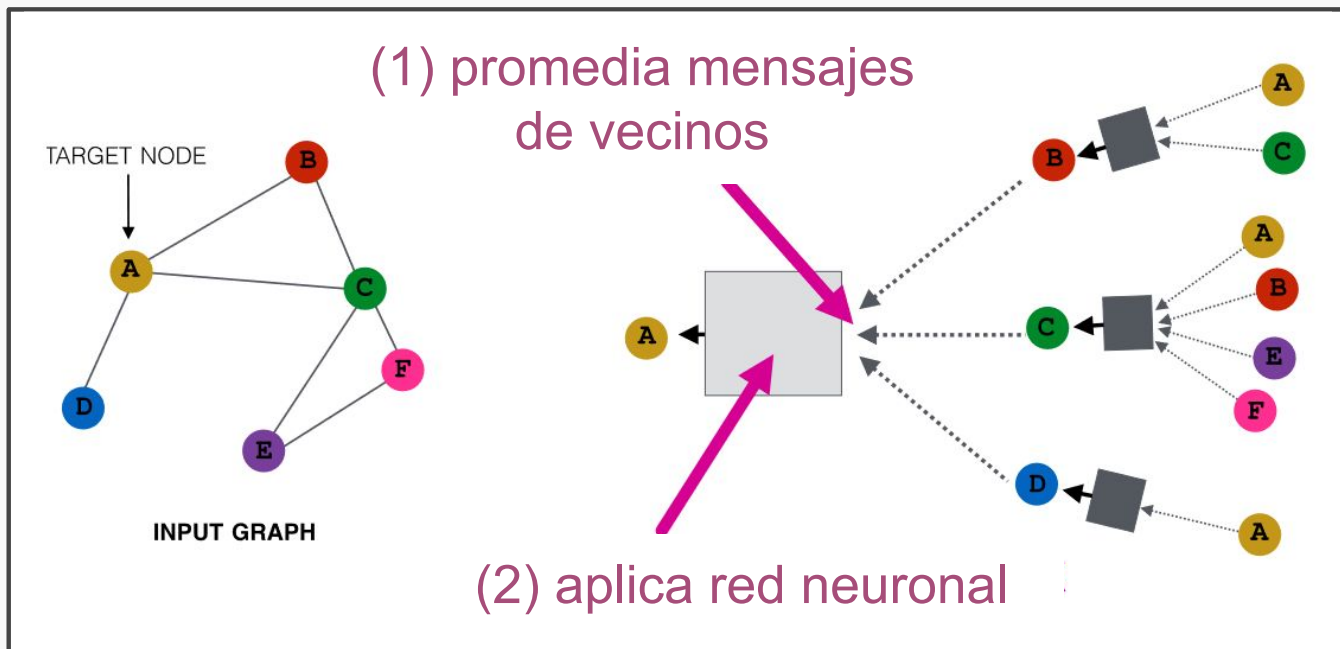
Agregación de vecindario

Las diferencias claves están en cómo diferentes soluciones agregan la información de los nodos al pasar por las capas del modelo:

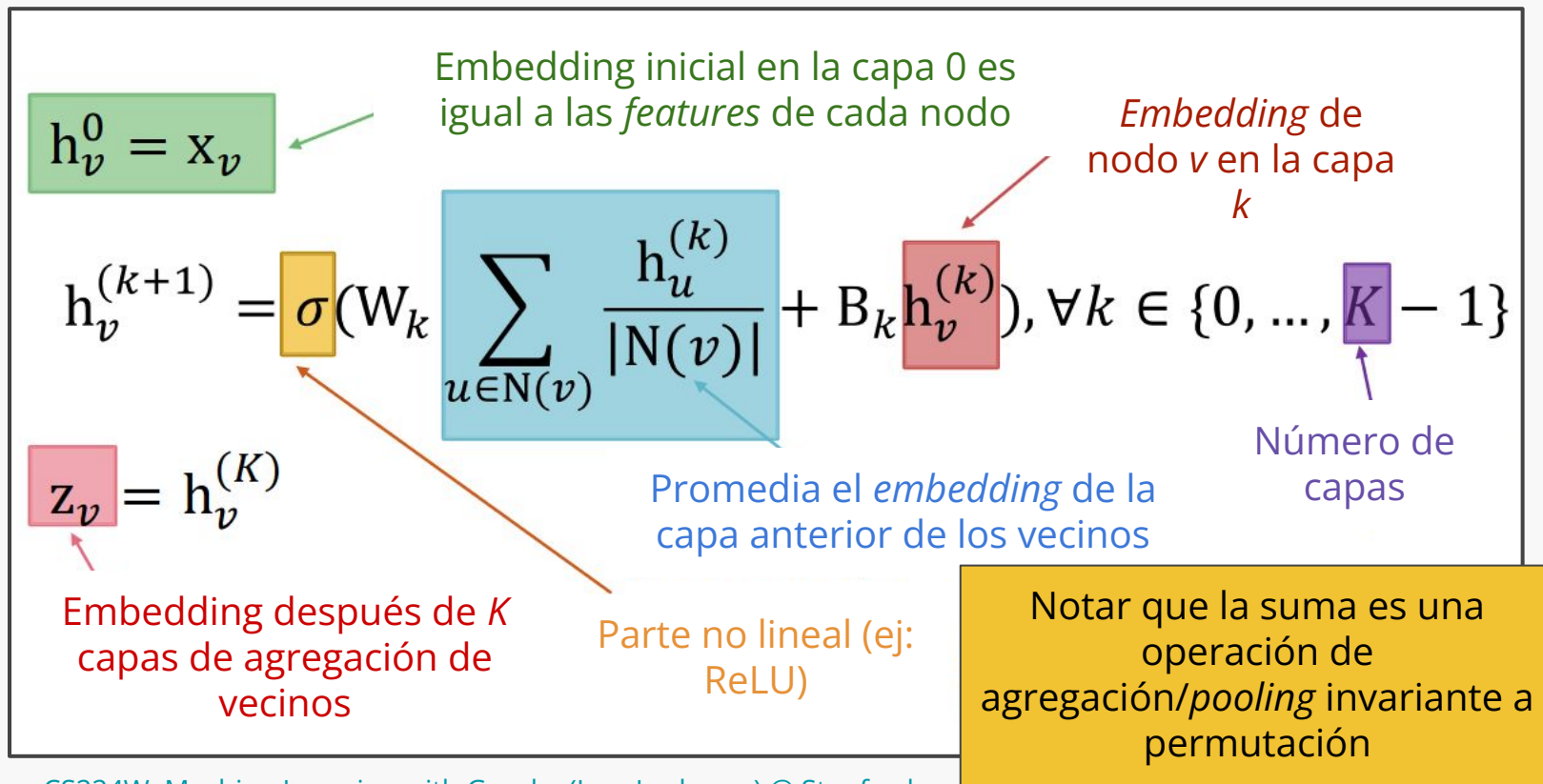


Agregación de vecindario

Idea simple: Promediar información de vecinos y luego aplicar una red neuronal



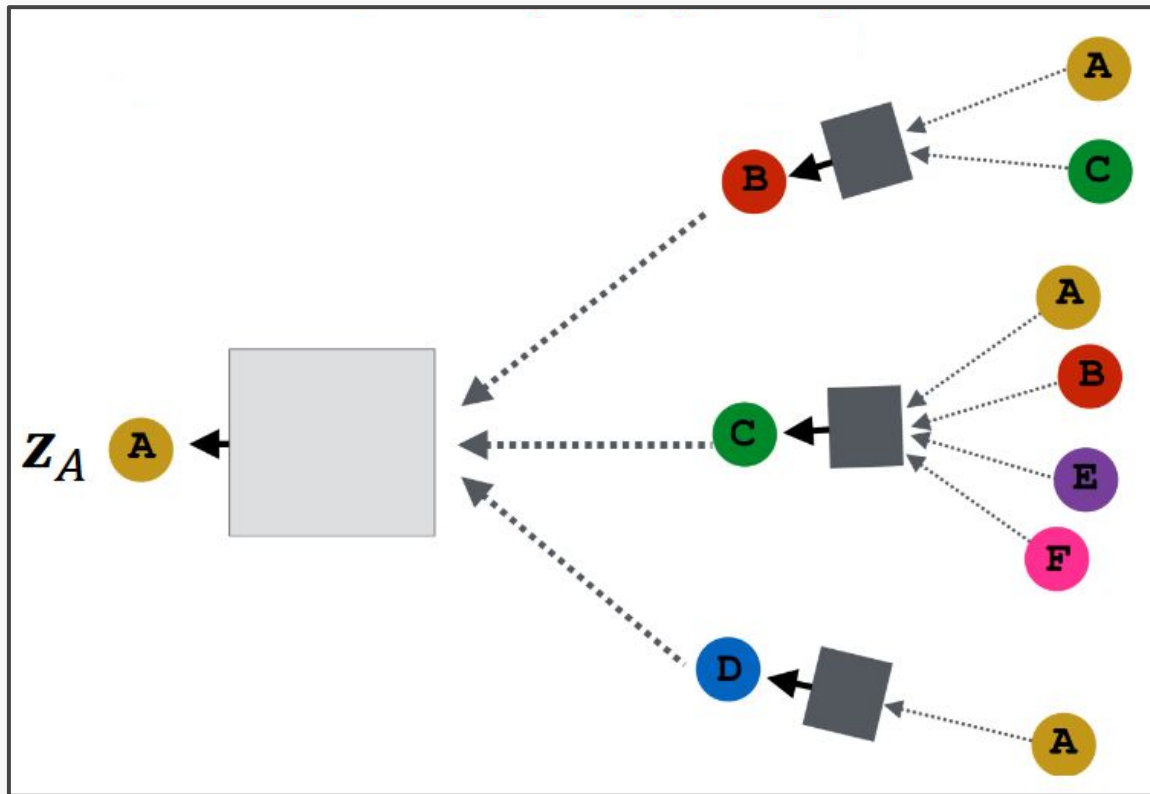
Formulación matemática



Entrenamiento del modelo

¿Cómo entrenamos la GCN para generar *embeddings* que funcionen?

Necesitamos definir una función de pérdida sobre los *embeddings*



Parámetros del modelo

h_v^k representación de nodo v en la capa k

Matrices de pesos entrenables
(esto es lo que aprendemos)

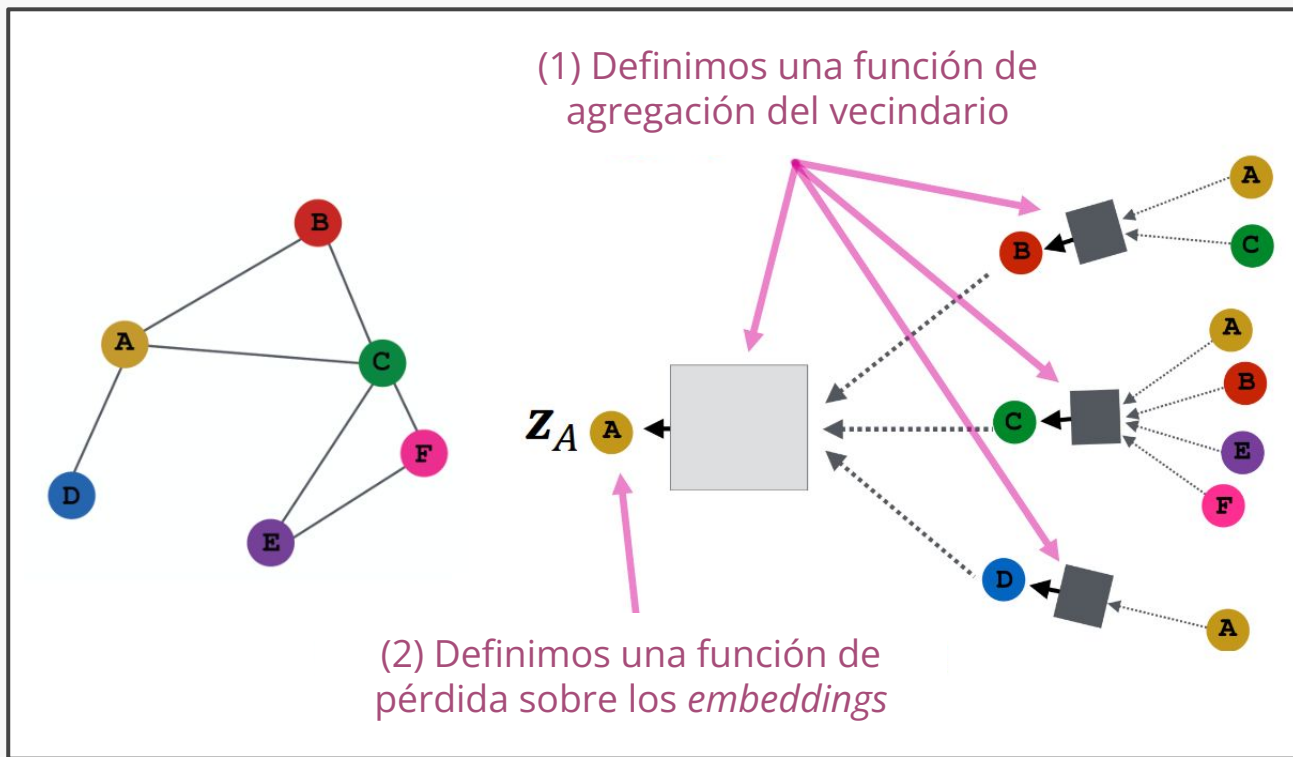
$$h_v^{(0)} = x_v$$
$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

$z_v = h_v^{(K)}$

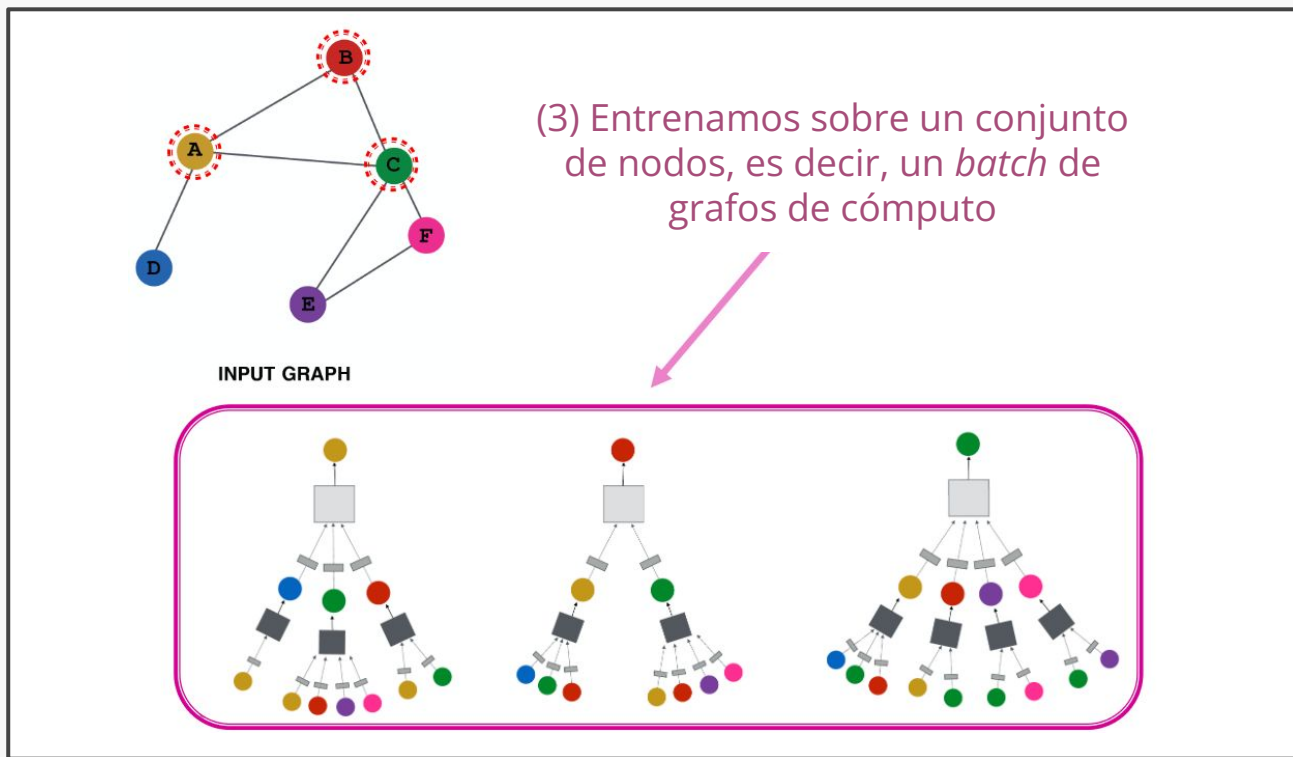
Embedding del nodo (salida final)

Podemos usar los *embeddings* en cualquier función de pérdida y utilizar SGD para entrenar

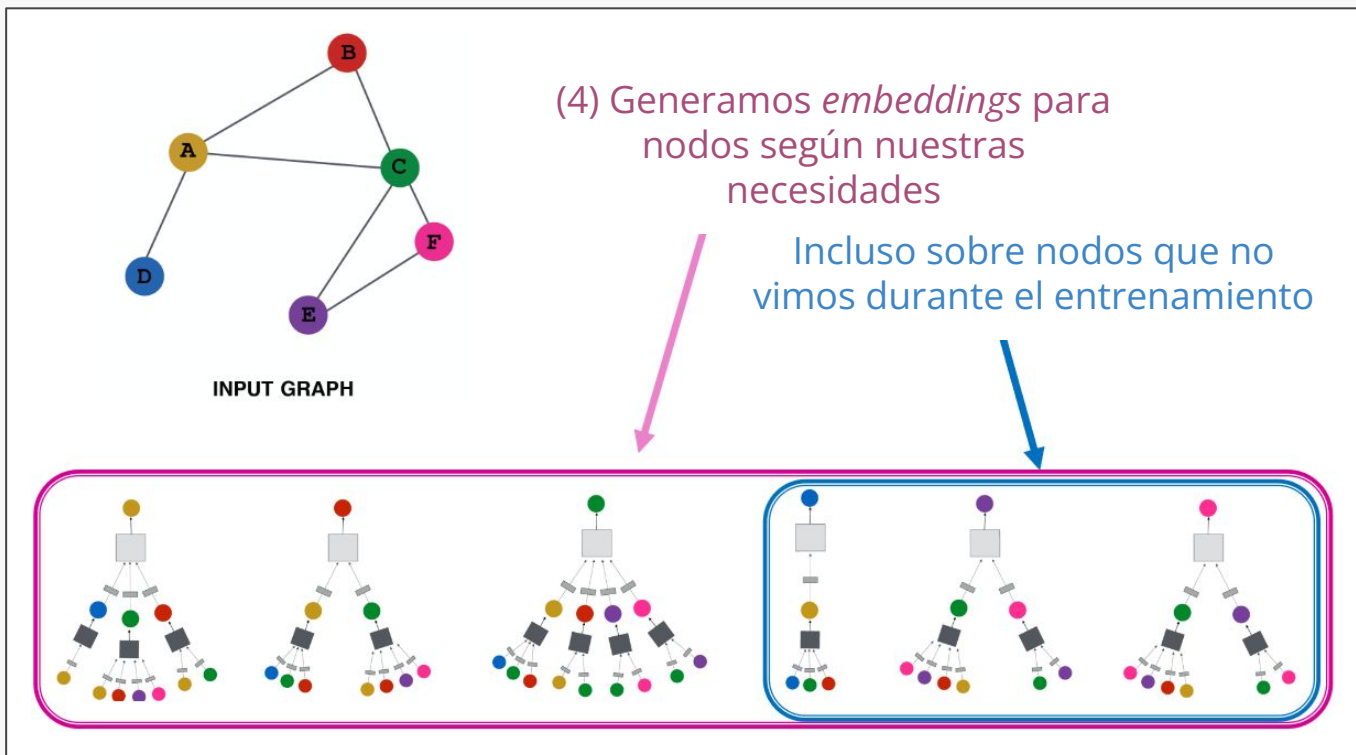
Cómo queda el modelo



Cómo queda el modelo

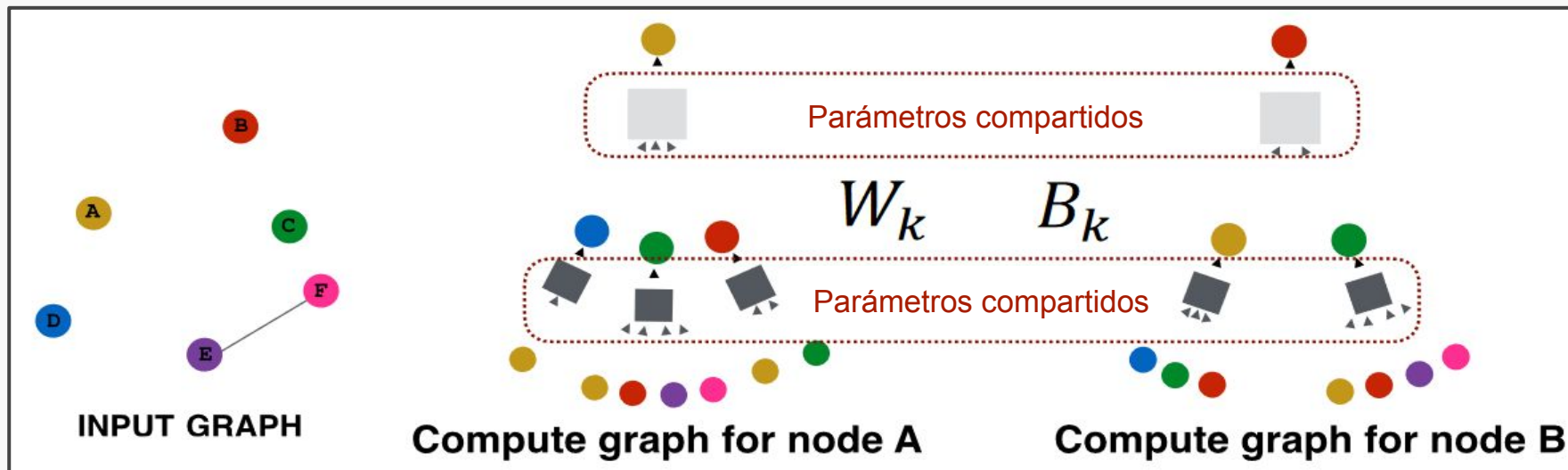


Cómo queda el modelo



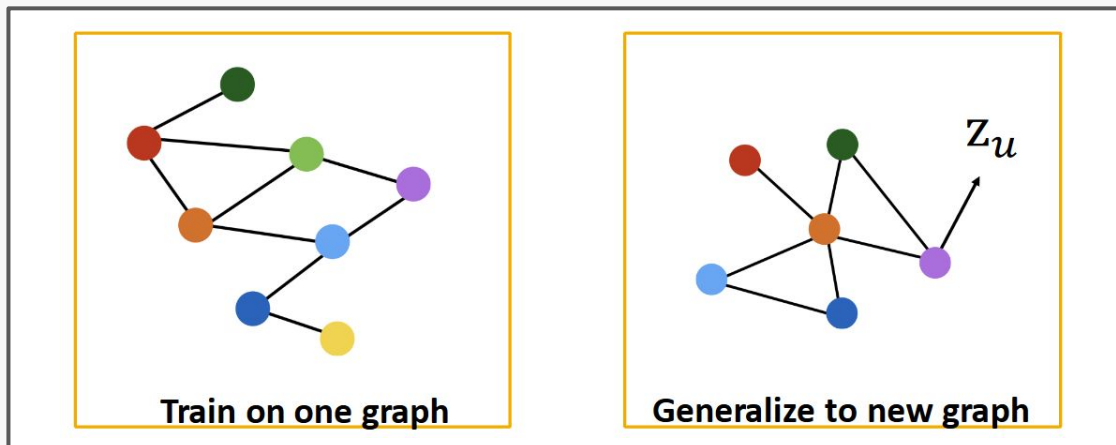
Capacidad inductiva (generalización)

Usamos los mismos parámetros para agregar todos los nodos



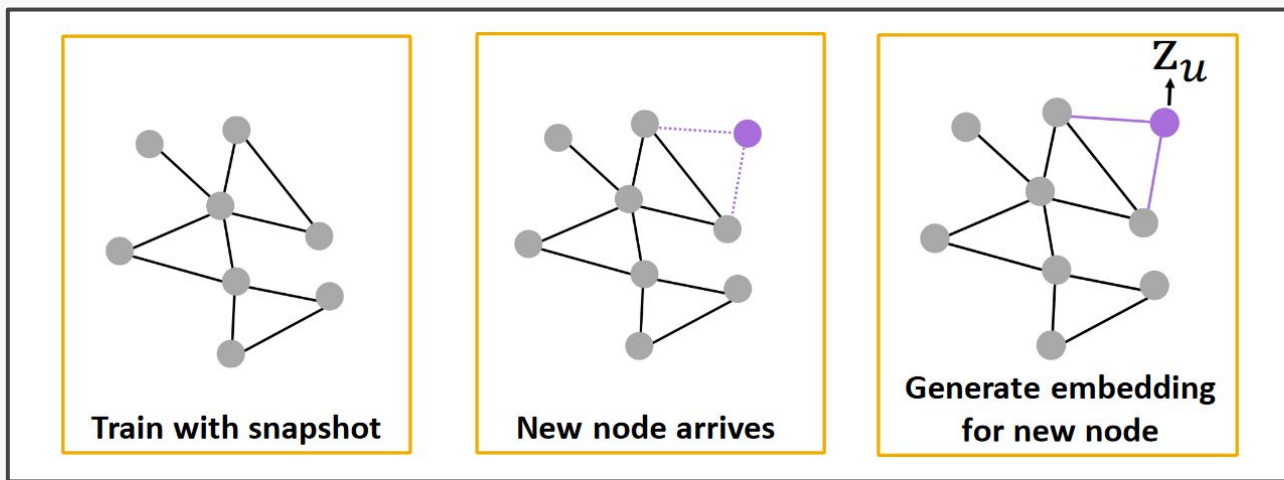
Capacidad inductiva (generalización)

Como podemos generalizar a nuevos nodos (y sus vecindarios), podemos generalizar a grafos completamente nuevos, que no hayan sido parte del entrenamiento (Ej: entrenar con interacción de proteínas de un organismo y generalizar a otro)



Capacidad inductiva (generalización)

Existen múltiples aplicaciones donde nos encontramos con nuevos nodos no vistos (Ej: video nuevo en un servicio, paper nuevo en una publicación, etc.), por lo que necesitamos generar nuevos *embeddings* para nuevos nodos



Clase 6.1

Deep learning para grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl